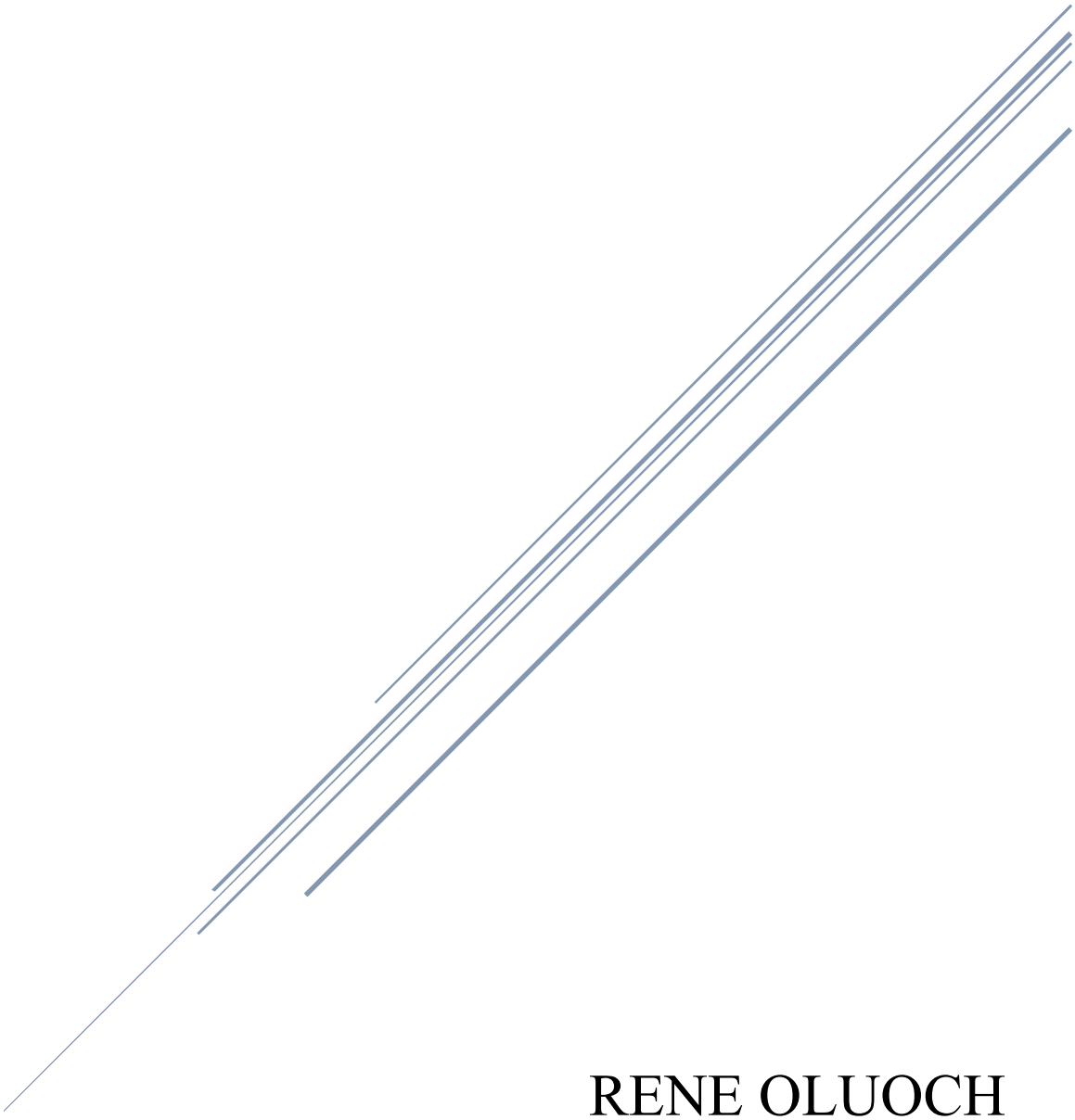
Microsoft Defender for DevOps

Integrating static analysis tools into the development lifecycle(Defender for DevOps)



RENE OLUOCH
CS-CNS09-25030

Contents

Introduction

I delved into Integrating static analysis tools into the development lifecycle (Defender for DevOps) with the objective of integrating static application security testing (SAST) tools directly into the DevOps lifecycle to strengthen code security during development. My focus was on using Microsoft Security DevOps and Bandit to identify and address vulnerabilities early in the CI/CD process. I began by importing a deliberately vulnerable Django-based web application into both GitHub and Azure DevOps, and then proceeded to set up Bandit locally for code scanning. I also configured Microsoft Security DevOps tools within Azure DevOps pipelines and GitHub Actions to automate security analysis. Through each stage, I explored how these tools analyze source code, flag insecure patterns, and embed security controls seamlessly into automated workflows — all without disrupting developer productivity.

Exercise 1: Importing the Vulnerable Code into GitHub and Azure DevOps

To begin the lab, I first imported the intentionally vulnerable web application that I would be analyzing and securing throughout the exercises. The source code is hosted on GitHub under the repository S2FrdQ/VulnDjango, which contains a Django-based web application intentionally designed with security flaws to facilitate security tool integration and testing.

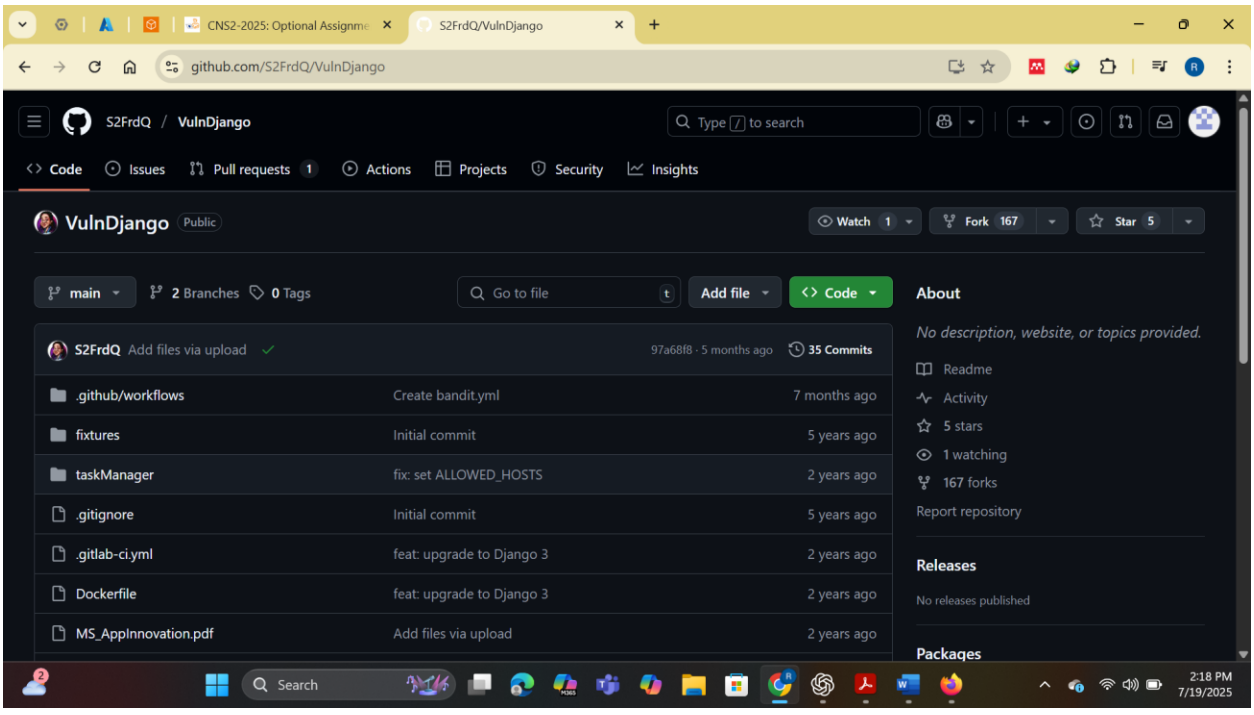


Fig 1.0 Locating the repository

I navigated to the GitHub repository and clicked on the **Fork** button to create a copy of the repository under my own GitHub account. This step was crucial as it allowed me to independently make changes and configure GitHub Actions without altering the original project.

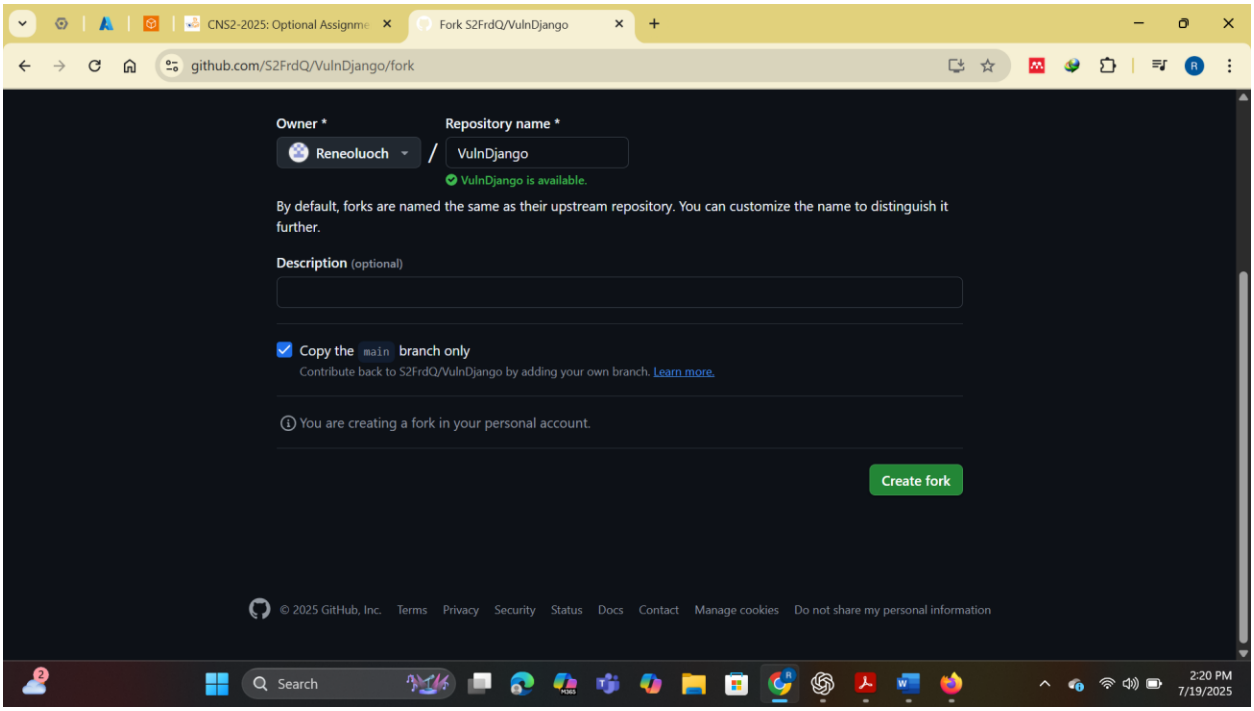


Fig 1.1 Creating a copy of the repository under my GitHub account

Next, I switched to **Azure DevOps** to prepare for integrating the code into a private development pipeline. After creating a devops account and signing into it, I created a **new private project** and named it VulnDjango to reflect the name of the vulnerable application.

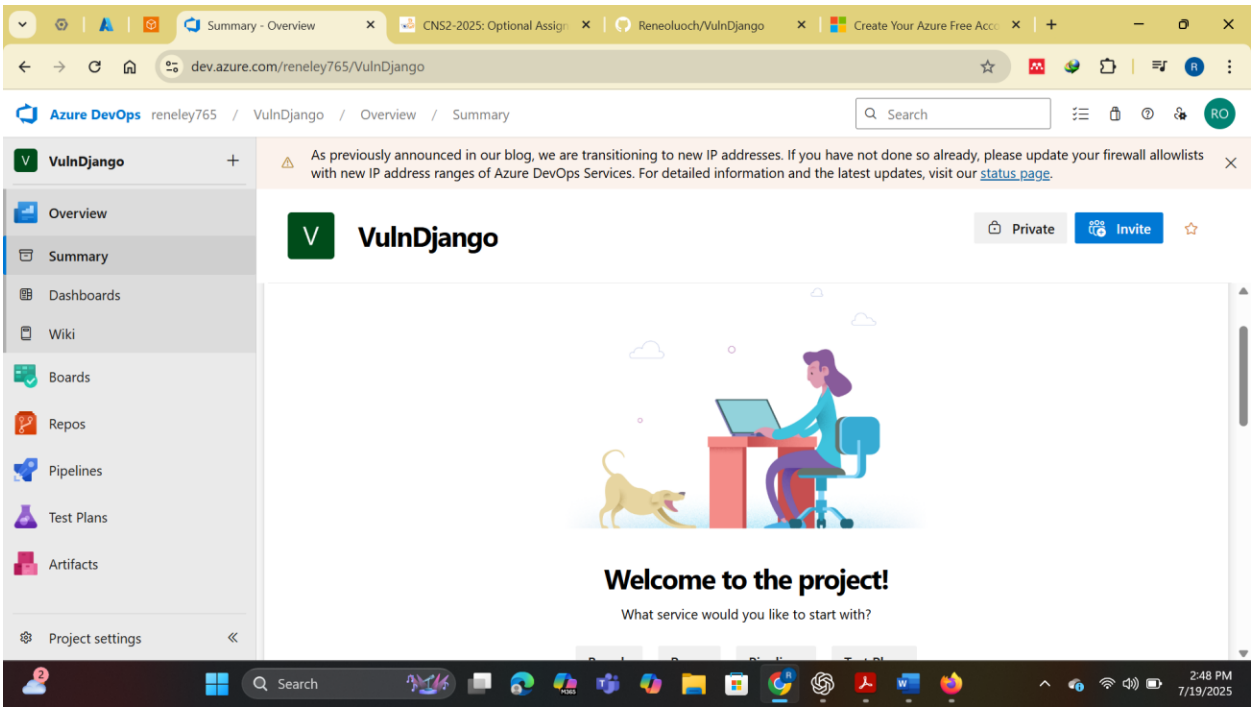


Fig 1.2 Creating project VulnDjango

Within the newly created project, I navigated to the **Repos** section and selected the option to **import a Git repository**.

In the import form, I entered the URL of the GitHub repository I forked earlier: https://github.com/S2FrdQ/VulnDjango.git

After initiating the import, Azure DevOps automatically pulled the entire codebase into the project. Once the process completed, I verified that all folders and files from the original Django application had been successfully cloned into my Azure DevOps environment. This completed the foundational step of setting up the vulnerable application in both GitHub and Azure DevOps, enabling me to carry out security scanning and DevOps pipeline configurations in the upcoming exercises.

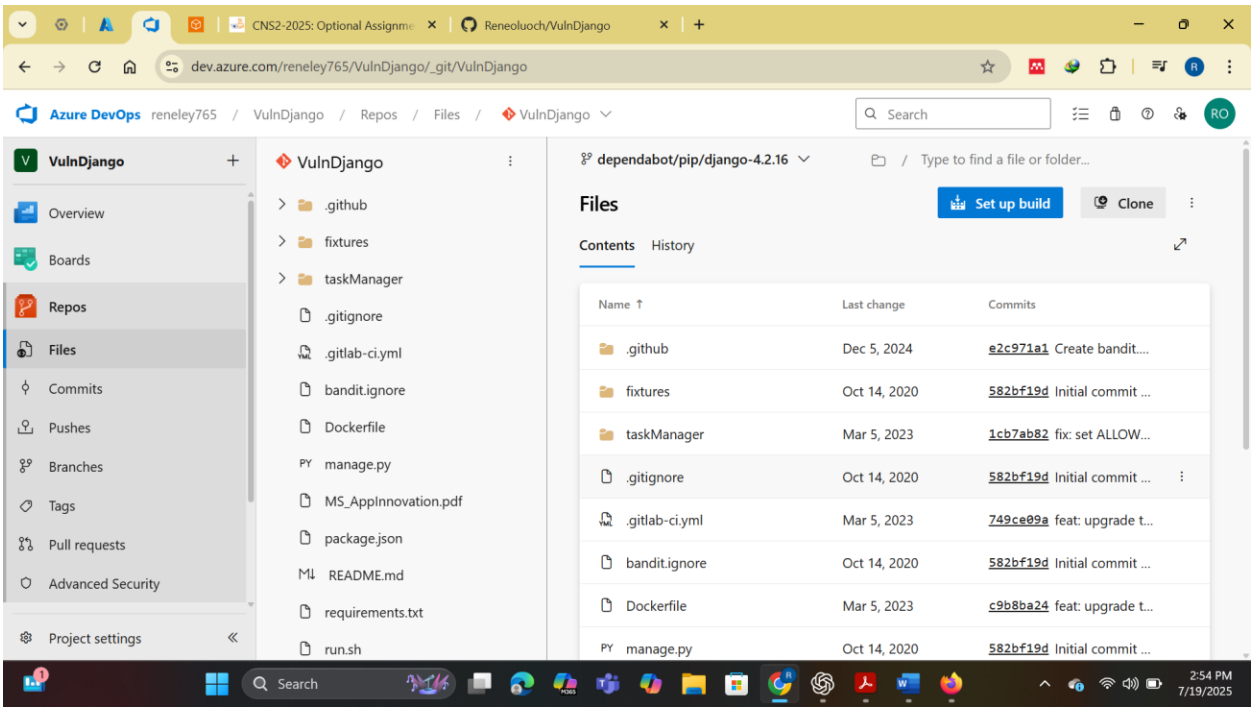


Fig 1.3 Successfully importing the git repository

Exercise 2:SAST scanning using Banditlocally.

After successfully importing the vulnerable Django codebase into both GitHub and Azure DevOps during Exercise 1, I proceeded to perform a local Static Application Security Testing (SAST) scan using **Bandit**. The purpose of this step was to identify potential security vulnerabilities embedded in the Python code before deployment, particularly those that might be exploited by attackers, such as hardcoded credentials, unsafe imports, or command injections. Since Django is written in Python, Bandit was a suitable and lightweight tool for source code analysis — it parses code files, constructs their Abstract Syntax Tree (AST), and applies a library of built-in security rules to detect common security issues.

I began by cloning the vulnerable source code repository locally on my Linux environment using the command git clone https://github.com/S2FrdQ/VulnDjango, which downloaded the web application into a directory named VulnDjango. After inspecting the directory structure, I navigated into the taskManager subdirectory using cd VulnDjango/taskManager, as this folder contains the core Django application files such as views.py, models.py, and settings.py — the appropriate targets for static analysis tools like Bandit.

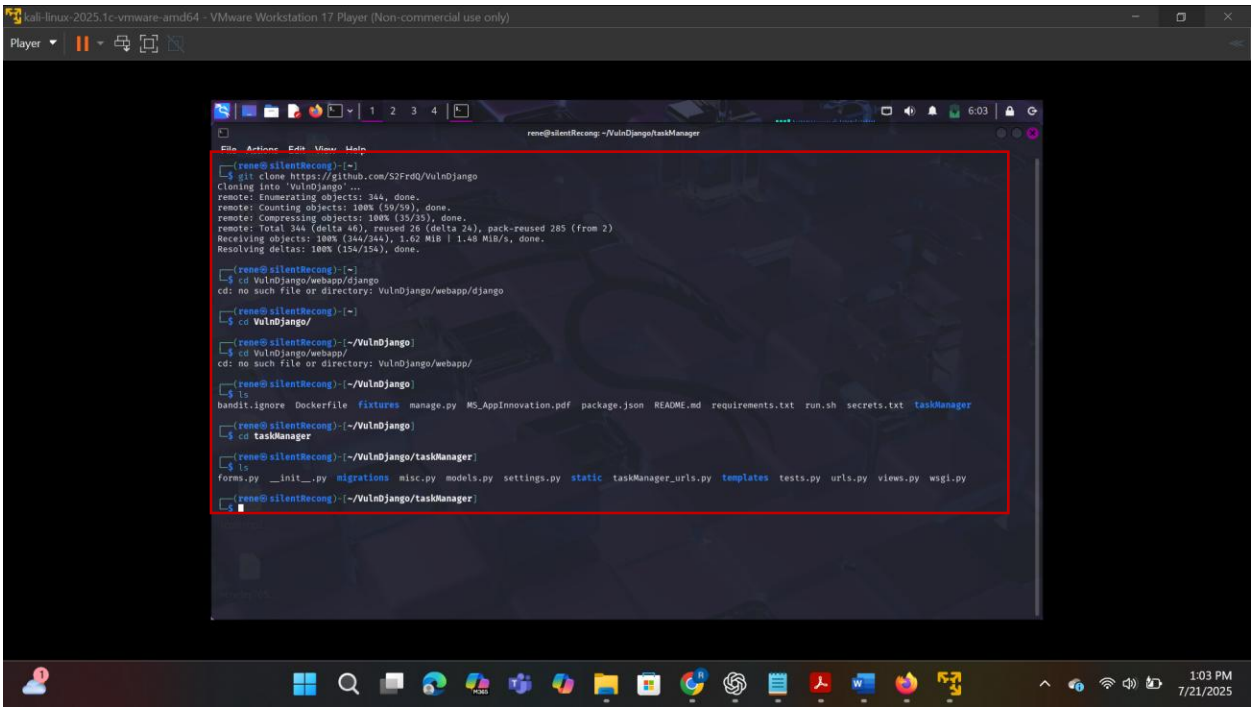


Fig 1.4 Cloning the repository and navigating to the application's folder

Next, I installed Bandit using pipx, a Python package manager that safely installs Python applications in isolated environments. I ran the command pipx install bandit, which completed successfully and made Bandit globally available without modifying the system Python environment. Since pipx handles PATH configuration automatically, I was able to run the tool immediately. To verify that Bandit was installed correctly, I executed bandit --help, which displayed the full list of available options—confirming that the tool was ready for use.

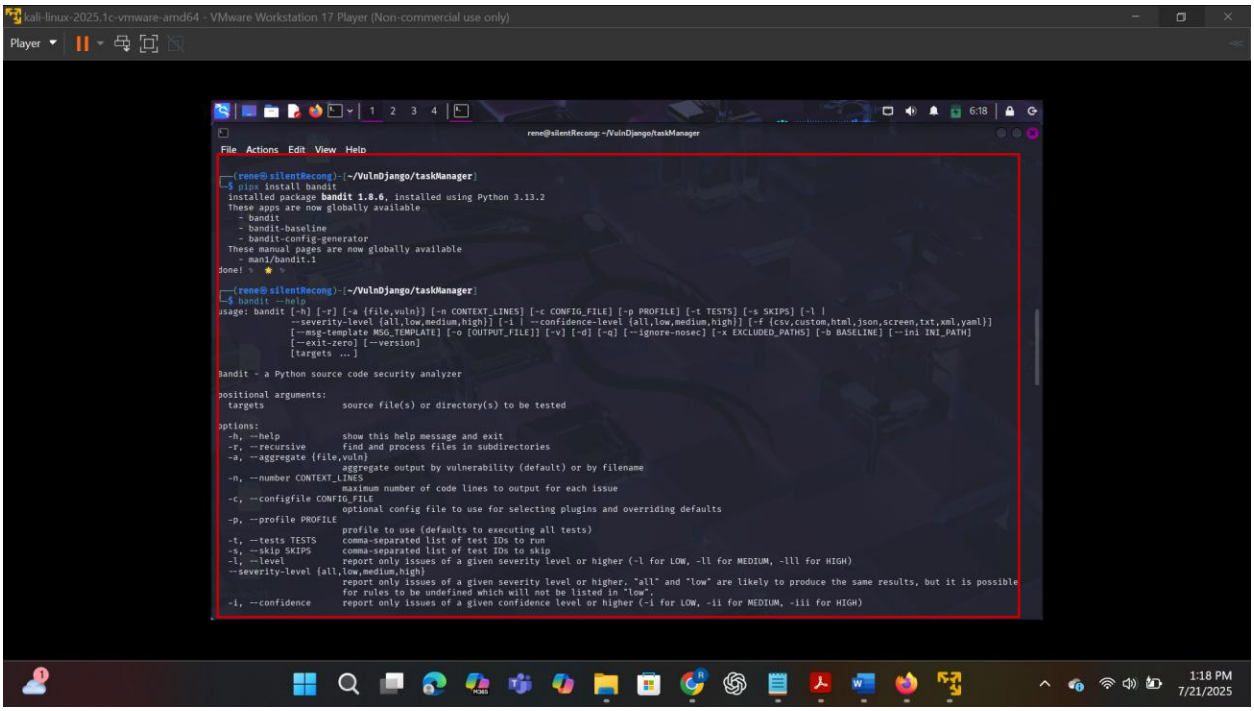


Fig 1.5 Installing bandit

With Bandit ready, I initiated the static scan from the django subdirectory by executing bandit -r ., which recursively scanned all Python files within the current directory and its subfolders. This produced a detailed console output listing security findings along with the severity and confidence level of each issue.

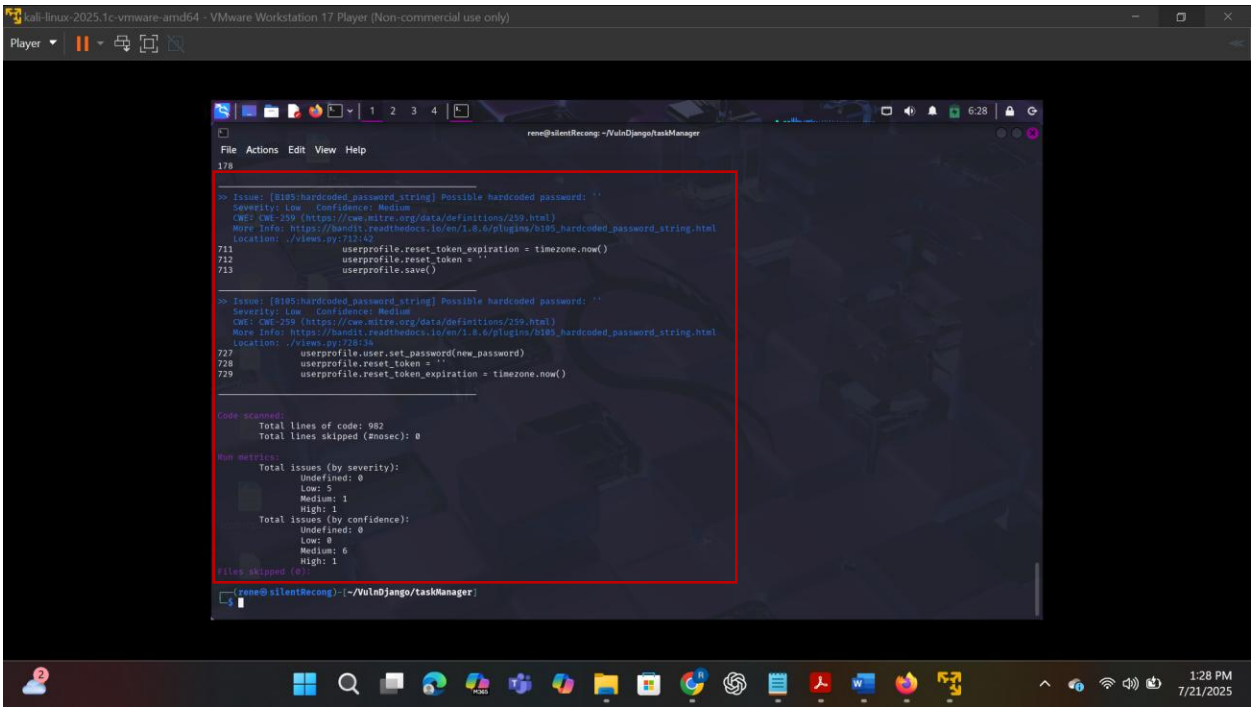


Fig 1.6 Running bandit -r .

To capture and store these results for later review, I appended the output with --format json | tee bandit-output.json, resulting in a complete command: bandit -r . --format json | tee bandit-output.json. This allowed me to view the scan results in real-time while simultaneously saving them in a structured JSON file.

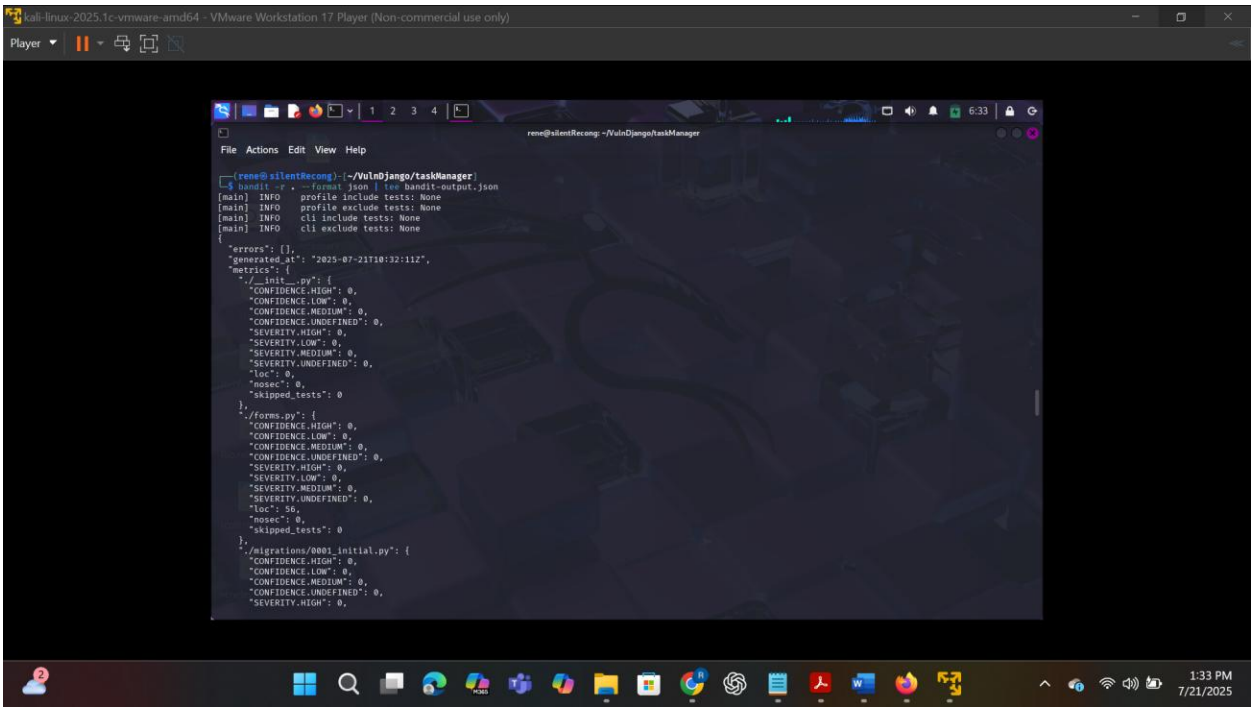


Fig 1.7 Running bandit -r . --format json | tee bandit-output.json

The generated bandit-output.json file contained detailed information about each security issue identified in the codebase. For every finding, Bandit reported the affected file, the specific line(s) of code involved, the nature of the vulnerability, its severity level (LOW, MEDIUM, or HIGH), and its confidence level in the result. For example, a **high-severity** issue was flagged in misc.py for using os.system() to execute a shell command with potentially unsanitized input — a known vector for shell injection attacks. Other findings included **hardcoded secrets** such as API keys and passwords in settings.py and insecure SQL query construction in views.py, which could lead to SQL injection vulnerabilities.

The scan completed in under a minute and immediately highlighted unsafe coding practices across the project. Bandit's output is especially useful because it not only identifies risky patterns but also links to relevant documentation and CWE references, helping developers understand the risks and recommended mitigations. While Bandit is highly effective at spotting common security pitfalls in Python code, it doesn't catch logical or contextual issues like flawed authentication logic — those require manual inspection and dynamic testing to uncover.

Running Bandit locally gave me a powerful first line of defense in identifying vulnerabilities before pushing any changes to CI/CD pipelines. Although this was a local scan, the results could easily be integrated into automated workflows in Azure DevOps or GitHub Actions to ensure continuous enforcement of secure code practices.

Exercise 3: Configuring the Microsoft Security DevOps Azure DevOps extension.

After completing the local Bandit scan in Exercise 2, I moved on to configuring Microsoft Security DevOps within Azure DevOps to enable continuous static analysis of my codebase directly in the CI/CD pipeline. This powerful extension automates the integration of multiple security analyzers, including Bandit, ESLint, CredScan, and Trivy, ensuring security checks are seamlessly incorporated into every code build. Before beginning the setup, I verified that I had administrator rights for my Azure DevOps organization, as these privileges are required to install organization-wide extensions.

I navigated to the Azure DevOps portal and opened the VulnDjango project I had previously created. To access the Marketplace, I clicked the shopping bag icon at the top right of the portal and selected “Browse Marketplace.” In the search bar, I typed “Microsoft Security DevOps” and selected the extension when it appeared in the results. On the extension’s information page, I clicked “Get it free,” then chose my Azure DevOps organization and confirmed the installation by clicking “Install.” Once the extension was added, I returned to the VulnDjango project and proceeded to set up a new pipeline to automate the security scanning.

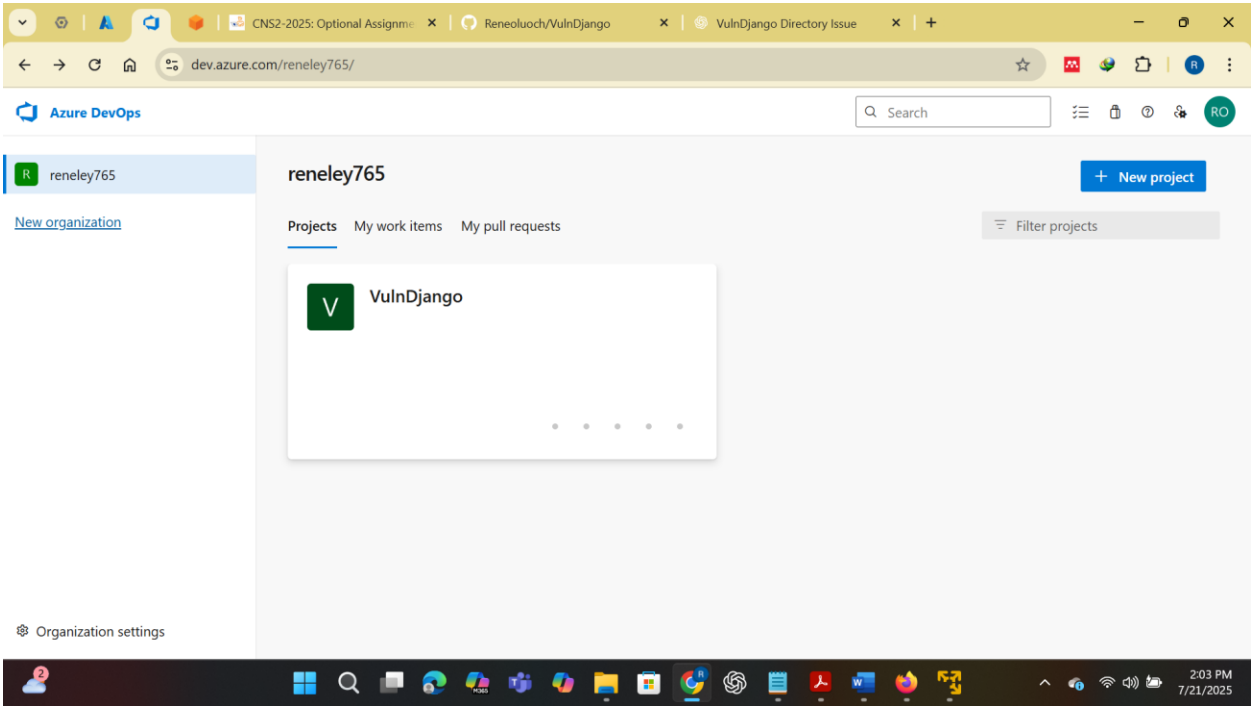


Fig 1.8 Activating microsoft devops extension

Within the project, I navigated to the Pipelines section, clicked on “New Pipeline,” and was prompted to answer where my code was stored. I selected “Azure Repos Git (YAML)” since the vulnerable Django code had already been imported into my repository. After selecting the VulnDjango repo, the system presented a YAML editor for configuring the pipeline. I replaced the default template with a custom YAML configuration tailored for Microsoft Security DevOps integration.

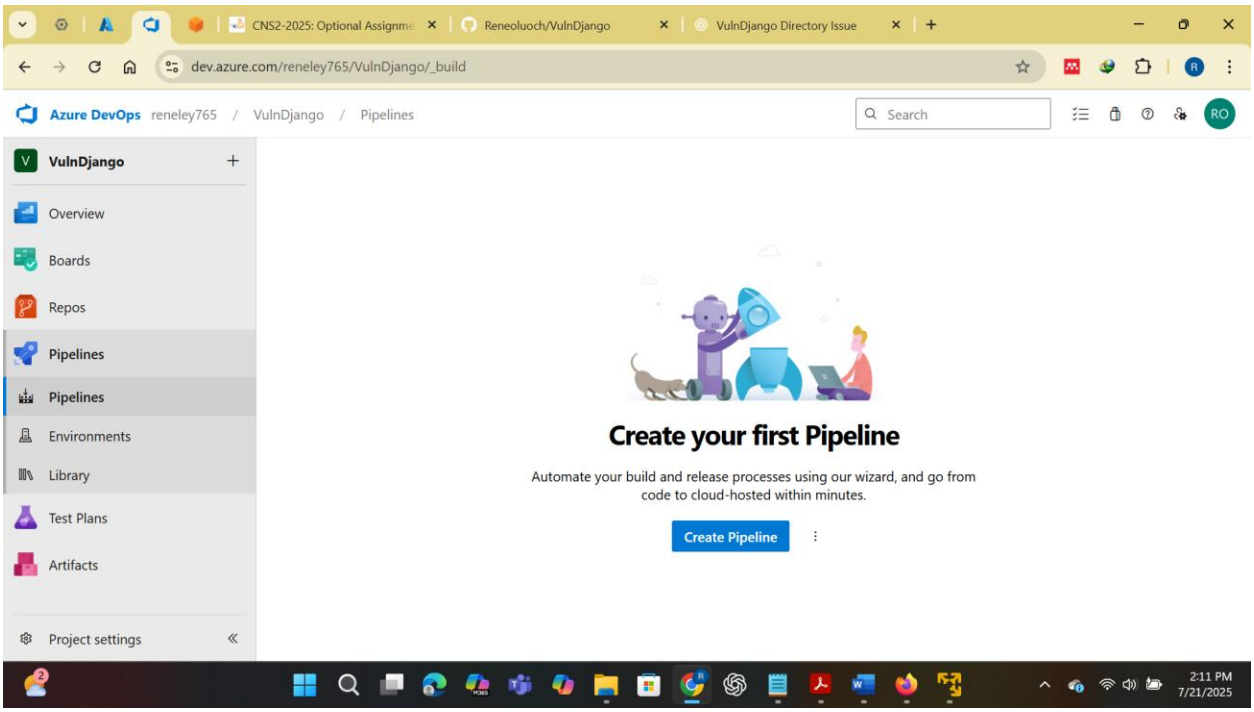


Fig 1.9 Creating the pipeline

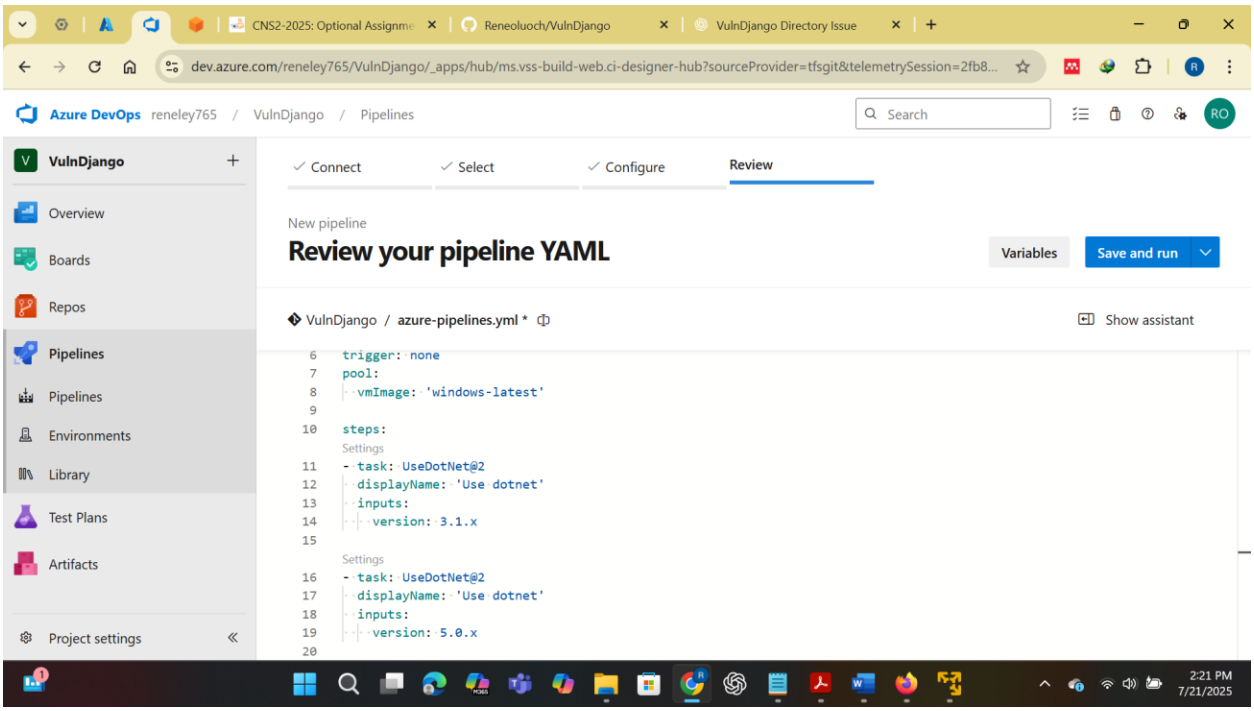


Fig 2.0 Editing the yaml file

The configuration I used looked like

Starter pipeline

Customized to support Microsoft Security DevOps + ESLint dependencies

trigger: none

pool:

  name: Default # Assuming you are using a self-hosted agent

steps:

- task: UseDotNet@2

  displayName: 'Use .NET Core 3.1'

  inputs:

    version: 3.1.x

```
- task: UseDotNet@2
  displayName: 'Use .NET 5.0'
  inputs:
    version: 5.0.x

- task: UseDotNet@2
  displayName: 'Use .NET 6.0'
  inputs:
    version: 6.0.x

# Install Node.js 18.x (compatible with ESLint 8.x)
- task: NodeTool@0
  displayName: 'Install Node.js 18'
  inputs:
    versionSpec: '18.x'

# Optional: Pre-install ESLint globally to avoid MSDO install issues
- script: |
    npm install -g eslint@8.56.0
  displayName: 'Preinstall ESLint (optional)'
  continueOnError: true

# Run MSDO after Node and ESLint are available
- task: MicrosoftSecurityDevOps@1
  displayName: 'Microsoft Security DevOps'
  name: msdo  # Name the step to access its outputs
```

```
- task: PowerShell@2
  displayName: 'Move SARIF to CodeAnalysisLogs folder'
  inputs:
    targetType: 'inline'
    script: |
      $sarif = "$(msdo.sarifFile)"
      Write-Host "SARIF file path: $sarif"
      New-Item -ItemType Directory -Force -Path CodeAnalysisLogs
      Copy-Item -Path $sarif -Destination "CodeAnalysisLogs/"


- task: PublishBuildArtifacts@1
  displayName: 'Publish CodeAnalysisLogs Artifact'
  inputs:
    PathtoPublish: 'CodeAnalysisLogs'
    ArtifactName: 'CodeAnalysisLogs'
```

I configured the pipeline to run static analysis using Microsoft Security DevOps on a self-hosted agent. The trigger is set to none for manual execution. Instead of using Microsoft-hosted agents, I specified a custom agent pool with pool: name: Default. To support analyzers that require .NET, I included steps to install multiple SDK versions using UseDotNet@2, targeting versions 3.1.x, 5.0.x, and 6.0.x. I then added a NodeTool@0 task to install Node.js version 18, which is required by some tools like ESLint. To prevent runtime errors, I followed up with a script step to globally install ESLint version 8.56.0. Finally, I invoked the main scanner with the MicrosoftSecurityDevOps@1 task, clearly labeled for easy identification. This configuration ensures that all dependencies are properly installed on the self-hosted agent, enabling reliable static code analysis during the pipeline run.

To set up a self-hosted agent for Azure Pipelines, and to bypass the error no hosted parallelism, I began by downloading the agent package from Microsoft's official site and saving it to my system. I then opened PowerShell and created a dedicated directory by running mkdir agent and navigating into it with cd agent. Inside the folder, I extracted the downloaded ZIP file using the command [System.IO.Compression.ZipFile]::ExtractToDirectory(...), which unpacked the agent into the working directory. Next, I configured the agent by executing .\config.cmd, which walked

me through authentication and connection to my Azure DevOps organization. Once configuration was complete, I optionally ran the agent interactively using .\run.cmd to confirm everything worked as expected. This setup allowed me to bypass hosted agent limitations and run Azure Pipelines on my own infrastructure.

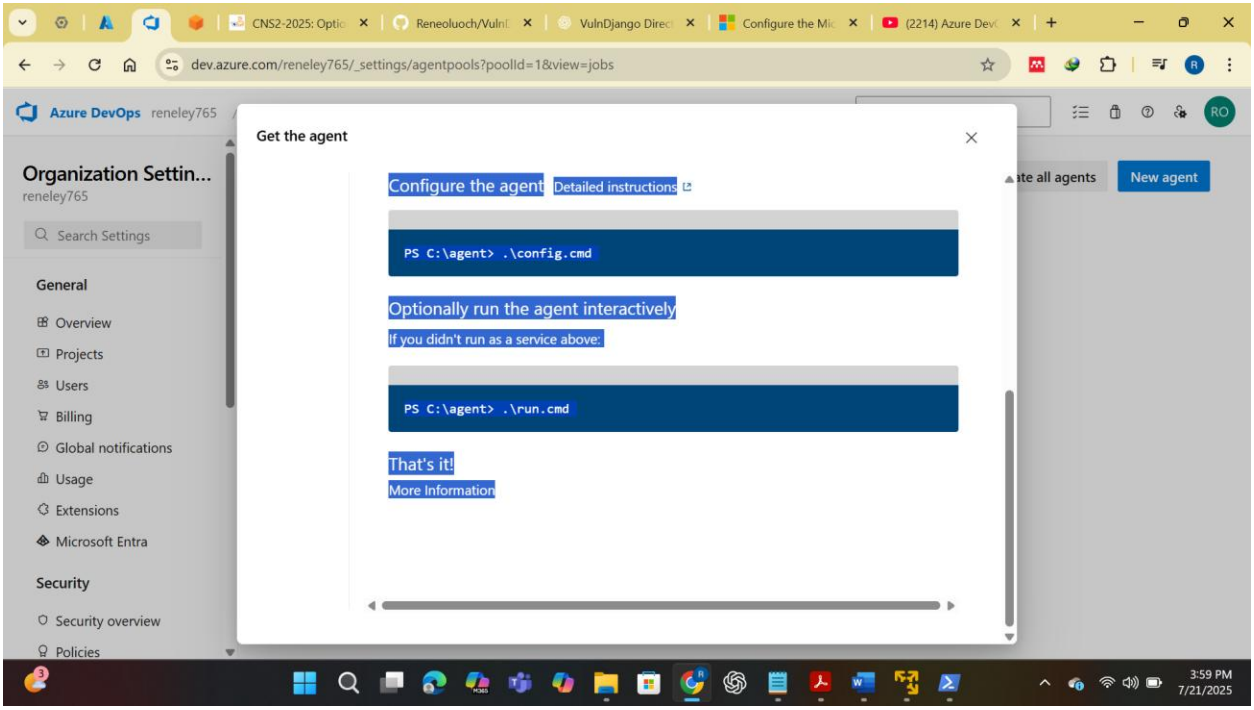


Fig 2.1 Setting up an agent to host locally

After finalizing the YAML, I clicked “Save and Run” to commit the configuration to the repository and execute the pipeline. The system redirected me to the running pipeline view, where I could monitor each task in real-time, including code checkout, environment setup, SDK installation, and finally the execution of security scans. Once completed, the results were available under the new “Scans” tab, introduced by the Microsoft Security DevOps extension. Here, I could view detailed vulnerability reports showing the severity, file path, line number, and remediation advice for each issue detected by the analyzers.

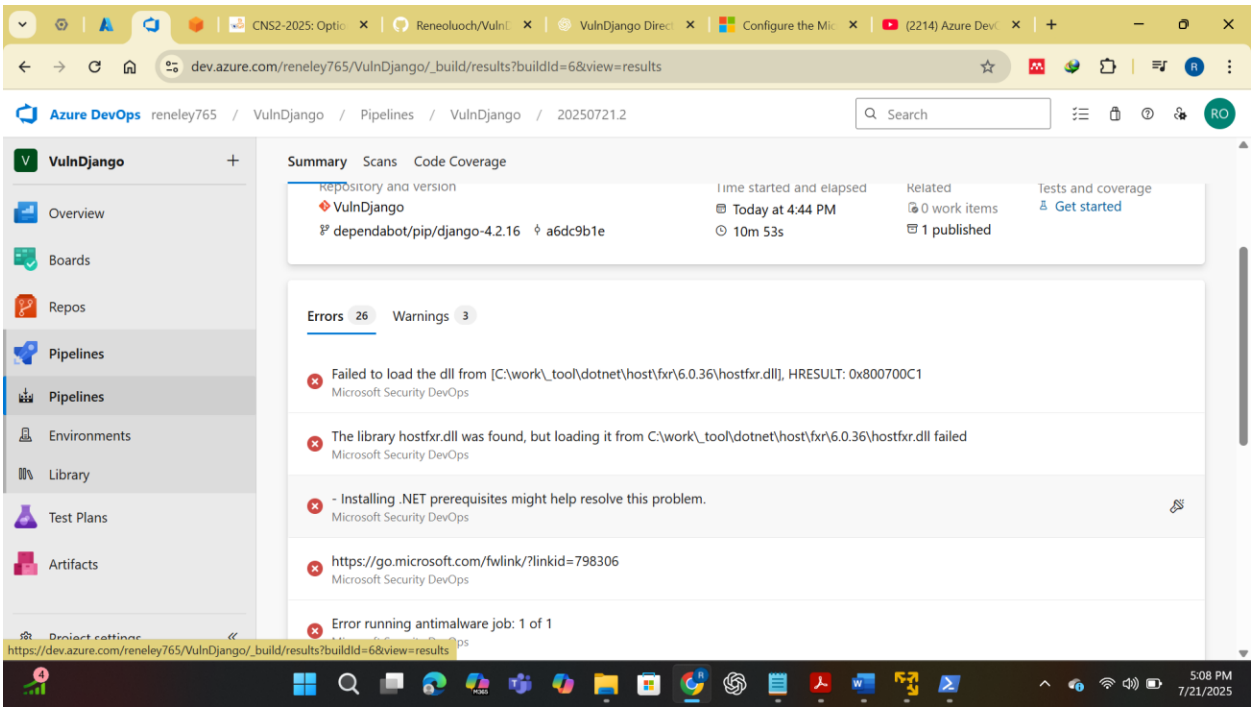


Fig 2.2 Configuration of the repository and executing the pipeline

To further enhance scan visibility, I installed an additional Azure DevOps extension called "SARIF SAST Scans Tab." This extension improves the rendering of scan results that are output in SARIF (Static Analysis Results Interchange Format), allowing for rich, structured display within the Scans interface. To install it, I again accessed the Marketplace, searched for "SARIF SAST Scans Tab," clicked "Get it free," selected my organization, and installed it. After doing so, any SARIF-based reports from Security DevOps or other tools like CodeQL were automatically presented in an interactive and readable format.

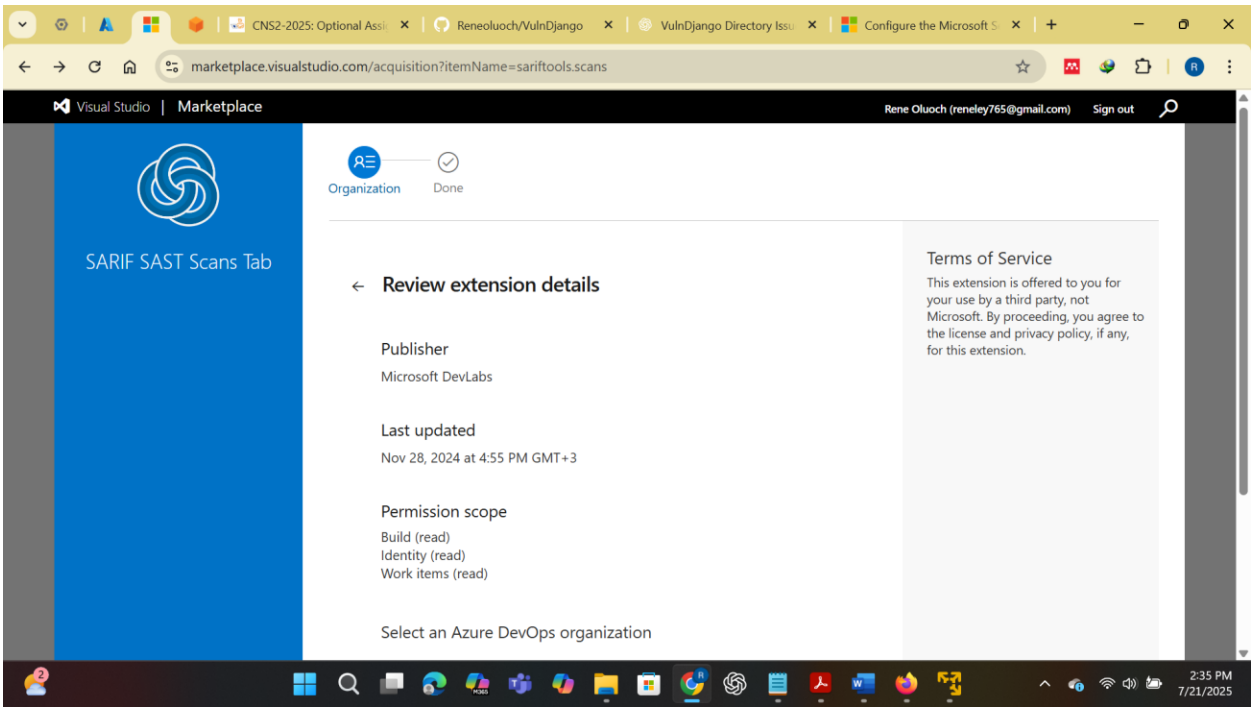


Fig 2.3 Installing SARIF SAST Scans Tab

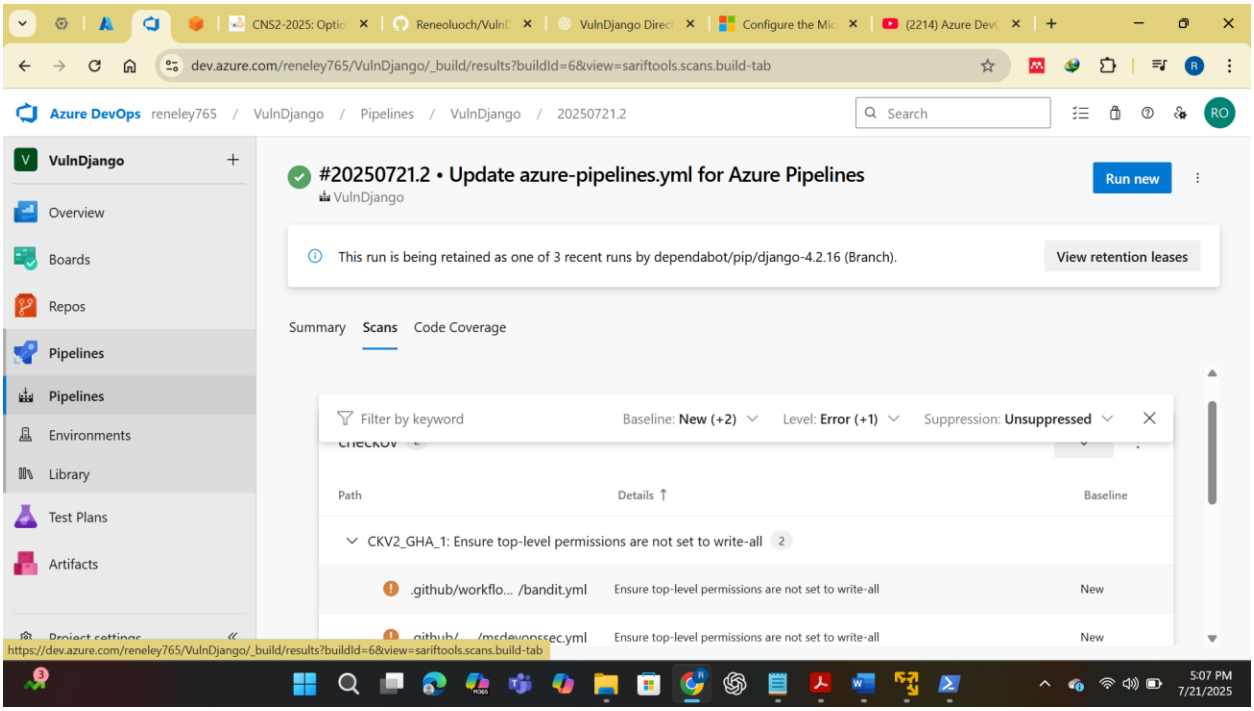


Fig 2.4 Rendered scan results of the output

By the end of this exercise, I had successfully configured a fully automated static application security testing (SAST) pipeline within Azure DevOps using Microsoft Security DevOps. This integration now ensures that every commit to the VulnDjango repository is automatically scanned for security issues, dramatically improving the security posture of the codebase and reinforcing a secure-by-design development workflow.

Exercise 4: Configuring the Microsoft Security DevOps GitHub actions

After completing the Azure DevOps pipeline setup, I proceeded to integrate Microsoft Security DevOps with GitHub Actions to enable similar static analysis directly within GitHub's CI/CD workflows. This setup ensures that every pull request and code push to the main branch of the repository is scanned automatically for vulnerabilities using Microsoft's static analysis tools, such as Bandit, CredScan, ESLint, and others.

I began by navigating to my forked copy of the VulnDjango repository on GitHub. Once inside the repository, I clicked on the **"Actions"** tab at the top of the interface to access GitHub's workflow automation features. Since no workflows had previously been configured, GitHub presented a "Get Started with GitHub Actions" screen, where I clicked the "**set up a workflow yourself**" option to create a new custom workflow from scratch. This action opened the online YAML editor where I defined the new pipeline.

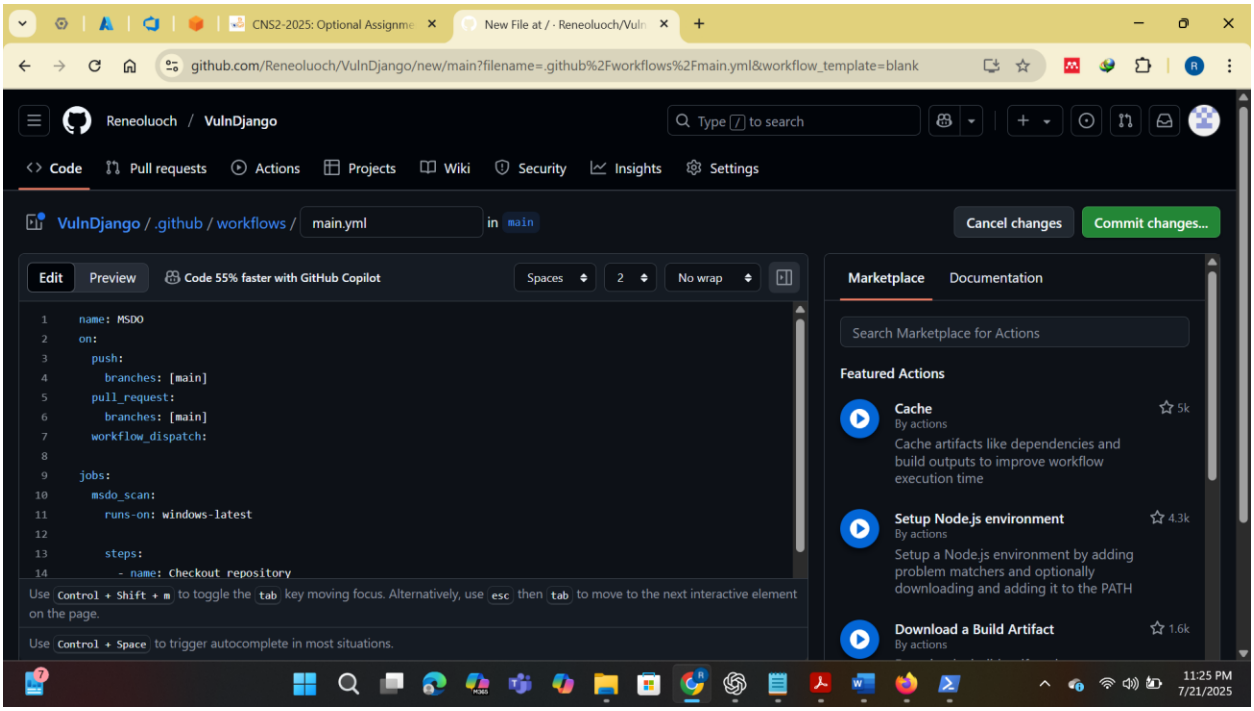


Fig 2.5 Editing the yaml file in github

In the filename textbox at the top, I entered a meaningful name for the workflow file — I chose msdevopssecgithub.yml to reflect its purpose. I then pasted the full GitHub Actions workflow configuration directly into the editor. The YAML script began by defining the workflow name using name: MSDO, and the triggers were set up to run the workflow on multiple events: on every push to the main branch, on any pull_request to the main branch, and also when manually triggered via the GitHub UI using workflow_dispatch.

The full configuration looked like this:

```
name: MSDO

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:


jobs:
  msdo_scan:
    runs-on: windows-latest


    steps:
      - name: Checkout repository
        uses: actions/checkout@v3


      - name: Setup .NET
        uses: actions/setup-dotnet@v3
        with:
          dotnet-version: |
            5.0.x
```

```
        6.0.x

    - name: Run Microsoft Security DevOps Analysis
      id: msdo
      uses: microsoft/security-devops-action@preview

    - name: Upload SARIF file to GitHub Security tab
      uses: github/codeql-action/upload-sarif@v2
      with:
        sarif_file: ${{ steps.msdo.outputs.sarifFile }}
```

This workflow ensures that the repository is first checked out (actions/checkout@v3), then it sets up the required .NET environment (actions/setup-dotnet@v3) to support the Microsoft tools. After that, the microsoft/security-devops-action@preview step runs the actual analysis across the codebase. Finally, the results are uploaded in SARIF format to GitHub's security dashboard using github/codeql-action/upload-sarif@v2.

After pasting the YAML into the GitHub editor, I clicked "**Start commit**," provided a brief commit message (e.g., "Add MSDO GitHub Action"), selected "**Commit directly to the main branch**," and clicked "**Commit new file**." Once the file was committed, GitHub automatically triggered the workflow based on the push event defined in the configuration. I returned to the "Actions" tab and saw the new MSDO workflow running under the list of actions. By clicking into the job, I was able to watch each step execute live, from repository checkout to scanning and uploading results.

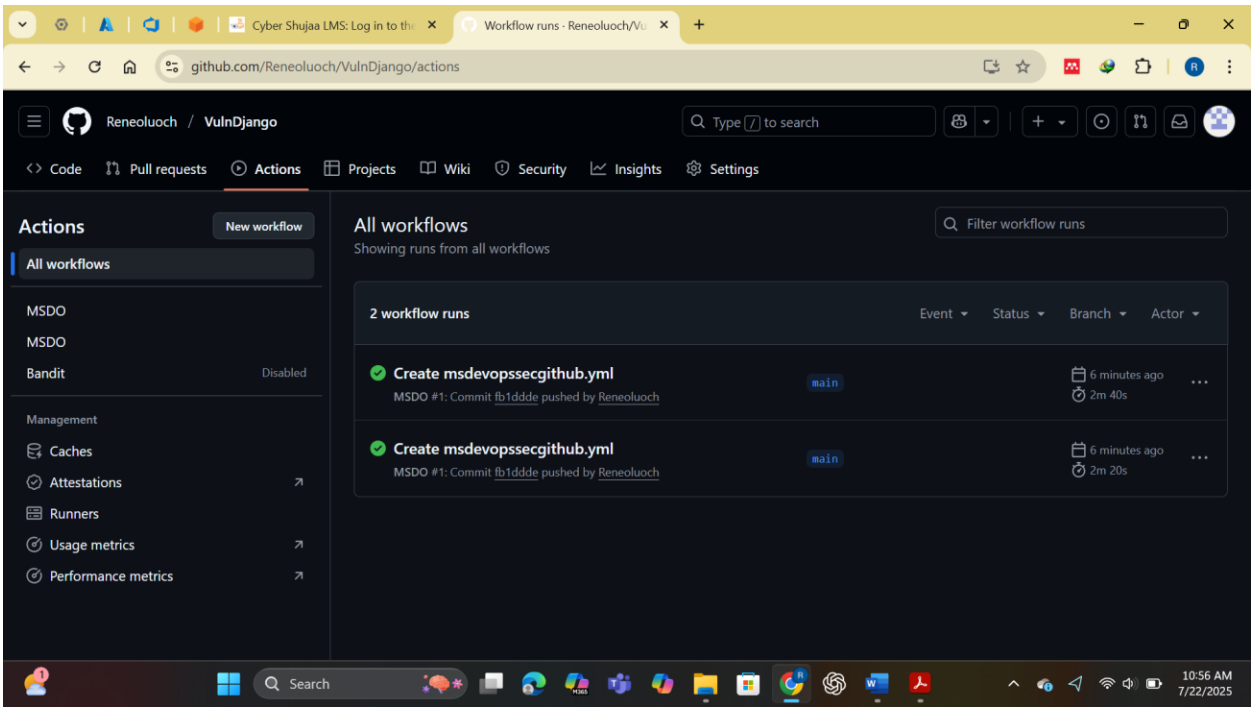


Fig 2.6 Successfully creating msdevopssecgithub.yml

To view the results of the scan after the workflow completed, I navigated to the "**Security**" tab within the GitHub repository and clicked "**Code scanning alerts**." Here, I selected "**Filter by tool**". This presented a list of all vulnerabilities and security issues detected by the scan, organized by severity, affected file, and line number. Each finding included detailed remediation guidance, making it easy to begin fixing issues directly from within GitHub.

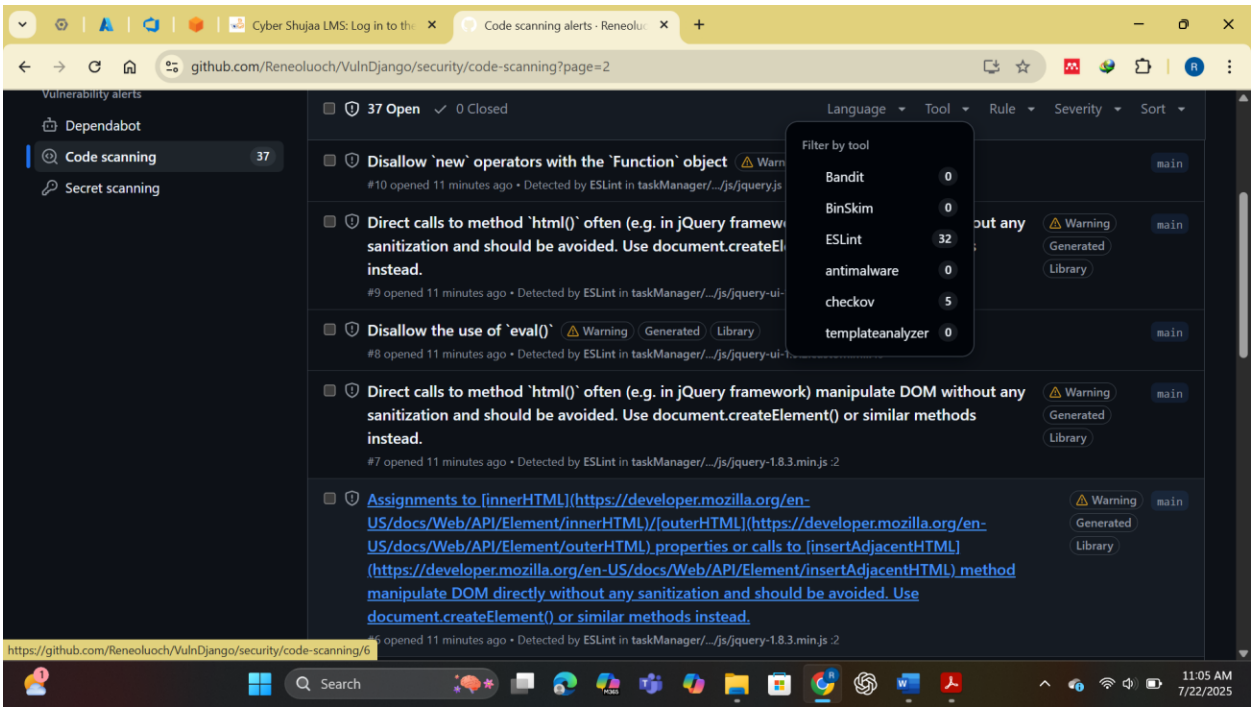


Fig 2.7 Filtering by tool

This GitHub Actions setup provides continuous security scanning for my project hosted on GitHub, complementing the Azure DevOps integration from earlier. The benefit of running scans in both platforms is twofold: it ensures broader security coverage across teams that use different repositories, and it demonstrates how Microsoft Security DevOps can integrate flexibly into both GitHub and Azure ecosystems. From this point forward, any code change or pull request made to the VulnDjango repository would be automatically analyzed, helping catch vulnerabilities early in the development cycle.

Exercise 5: DevOps Security Workbook

After completing the integration of Microsoft Security DevOps into the Azure DevOps pipeline and successfully running automated SAST scans, I moved on to **exploring the DevOps Security Workbook** available within Microsoft Defender for Cloud. This workbook provides centralized visibility into the security posture of DevOps environments by aggregating scan results, repository configurations, permission models, and policy compliance. It's a critical resource for understanding and remediating risks across CI/CD workflows.

To begin, I navigated to the **Azure Portal** and selected **Microsoft Defender for Cloud** from the left-hand menu. Within the Defender for Cloud dashboard, I scrolled down to the **Workbooks** section. Here, I located and opened the **"DevOps Security (Preview)"** workbook, which is

specifically designed to analyze and visualize DevOps security data from both **Azure DevOps** and **GitHub** sources.

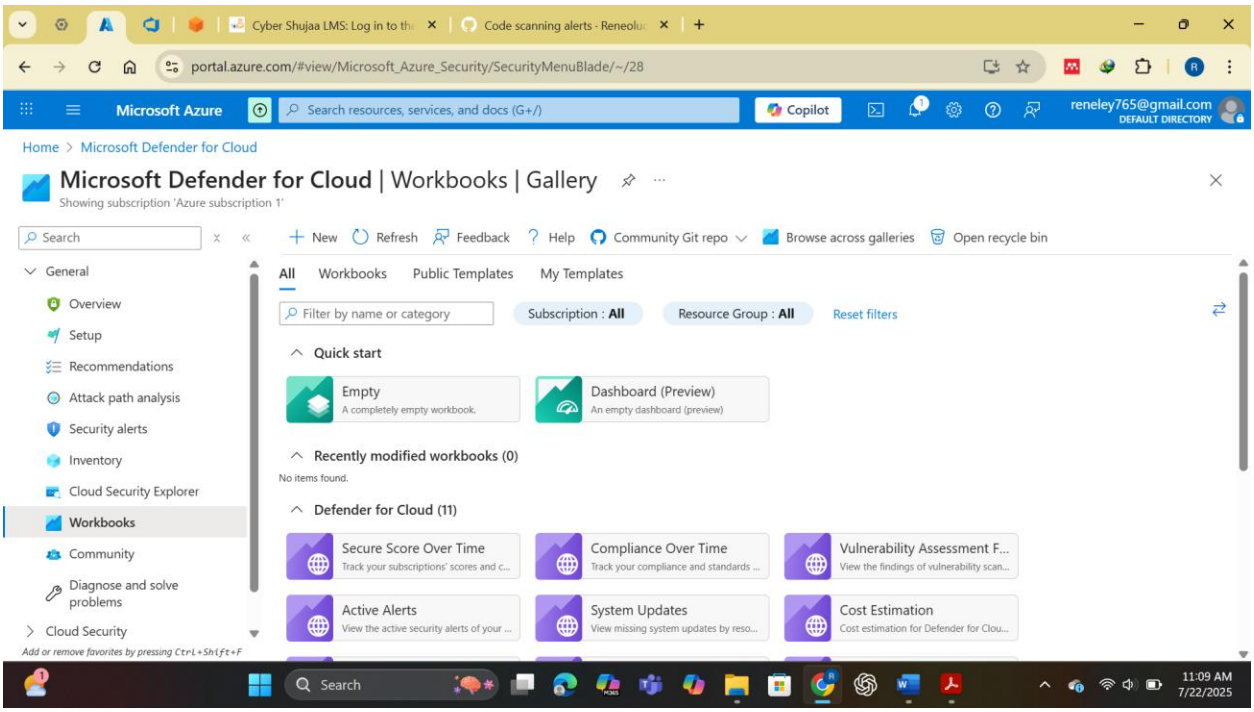


Fig 2.8 Workbook pane

Once the workbook was open, I allowed it a few moments to load and process telemetry from my linked DevOps environment. The workbook pulled in findings generated from the Microsoft Security DevOps scans that were previously executed via the Azure DevOps pipeline. This included results from tools like Bandit, CredScan, and Trivy — all of which were automatically translated into actionable insights.

The dashboard offered rich visualization panels and filters, making it easy to drill down into specific security issues across repositories and pipelines. For instance, I could view graphs showing the **distribution of vulnerabilities by severity**, such as Critical, High, Medium, and Low, as well as breakdowns of affected repositories, offending policies, and contributing risk factors. The **"Code Analysis Summary"** section gave a concise overview of all SAST issues identified, including the specific tools that flagged them and links to trace the exact pipeline job or GitHub Action run that produced the alert.

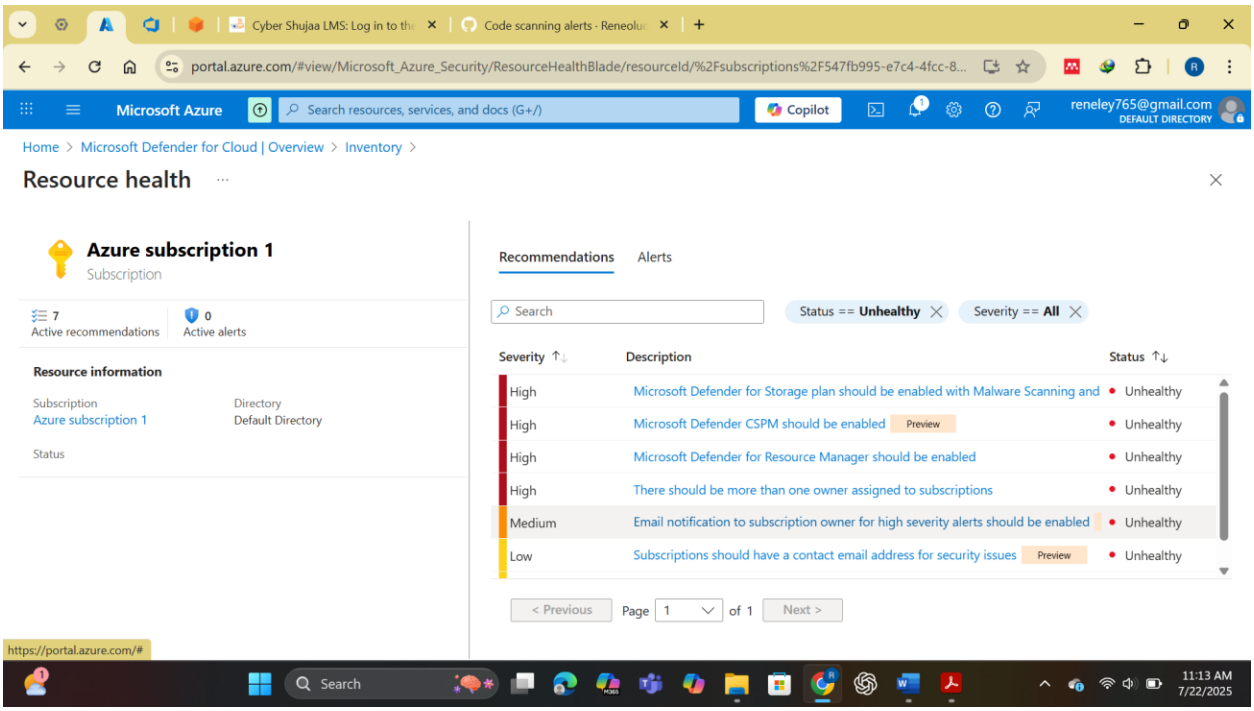


Fig 2.9 Resource information

Another useful feature was the **"Risky Permissions"** section, which highlighted repository and pipeline access configurations that could enable lateral movement or privilege escalation. This provided a higher-level risk assessment that goes beyond just code issues — it looked at identity and access design, misconfigurations, and exposure of critical secrets or tokens.

I also found the **"Policy Compliance"** area insightful, where I could track compliance against custom or built-in Azure policies that govern secure DevOps practices. Examples included policies requiring all pipelines to include security scanning steps, prohibiting use of overly broad permissions, and enforcing secure branching models.

The DevOps Security Workbook served as an excellent centralized dashboard for interpreting and responding to scan results and security posture indicators across my CI/CD infrastructure. It not only provided full visibility into static analysis findings but also correlated that information with broader DevSecOps concerns such as policy violations, permissions risks, and compliance posture. Leveraging this workbook is crucial for any team looking to build a secure and resilient DevOps pipeline, as it turns raw scan data into strategic guidance for remediation and improvement.

Conclusion

By working through this task, I gained valuable hands-on experience in applying static analysis tools within a DevOps environment. I discovered how Bandit quickly highlights common Python

security flaws, and how Microsoft Security DevOps integrates multiple scanning tools into build pipelines with minimal setup. I also appreciated the visibility that tools like Defender for Cloud provide, enabling teams to monitor scan results and respond proactively. Overall, this exercise helped me better understand the practical steps involved in embedding security into development pipelines, and reinforced the importance of making secure coding an integral part of the DevOps process from the very beginning.